1 ХІД РОБОТИ

1.1 Дослідницьке поле

Дослідницьке поле роботи – дослідження продуктивності та надійності диспетчеризації віртуальних потоків на платформі JVM в умовах взаємодії з нативним кодом (FFM та JNI) і блокуючими операціями вводу-виводу. Англійською: Performance and scheduling reliability analysis of JVM virtual threads under native boundary interactions (FFM and JNI) and blocking I/O workloads.

Вибір поля обумовлено особистим інтересом до внутрішньої архітектури віртуальних машин, розробки високонавантажених серверних систем на Java/Kotlin та актуальністю переходу на віртуальні потоки (Project Loom) разом із новим інтерфейсом взаємодії з нативною пам'яттю (Panama FFM API).

Для обрання дослідницького поля виконувався пошук за темами, що відповідали моєму технічному стеку та сфері наукових інтересів (Java/Kotlin, багатопоточність, рантайми, системна оптимізація), по таких наукових репозиторіях і базах даних, як ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar, а також інженерних репозиторіях OpenJDK (JEPs). Джерела, використовувані надалі, було також знайдено через них.

1.2 Ключові проблеми

1.2.1 Методологія

Для формування бібліографічної бази та аналізу предметної області використовувався комбінований підхід: пошукова система Perplexity для первинної навігації та модульний булевий (бінарний) пошук у наукометричних базах (ACM Digital Library, IEEE Xplore, ScienceDirect, Springer, arXiv, Google Scholar). Перелік пошукових запитів:

- Базова поведінка диспетчера та масштабованість:
  - ”Virtual Threads” AND ”carrier starvation”
  - ”Project Loom” AND ”tail latency”
  - ”Virtual Threads” AND ”scalability”
- Нативний пінінг та міграція (JNI / FFM API):
  - ”Virtual Threads” AND ”carrier pinning” AND ”JNI”
  - ”Virtual Threads” AND ”FFM” AND ”pinning”
  - ”Foreign Function Memory” AND ”JNI” AND ”benchmark”

- Еволюція моніторів синхронізації (JEP 491 / JDK 24+):

– ”Virtual Threads” AND ”JEP 491”
– ”Virtual Threads” AND ”synchronized” AND ”pinning”
– ”residual pinning” AND ”JDK 24”

- Міжрівнева телеметрія та моделювання:

– ”Virtual Threads” AND ”eBPF” AND ”scheduler”
– ”Virtual Threads” AND ”queueing model”
– ”Virtual Threads” AND ”JFR” AND ”pinning”

1.2.2 Знайдені ключові джерела

1. FCoral: A Framework for Fine-grained Concurrency Evaluation Across Programming Language Runtimes. 2026. (https://doi.org/10.1145/3847668).
2. Misleading Microbenchmarks on the Java Virtual Machines. 2026. (https://doi.org/10.1145/3748522.3779882).
3. Practical Benchmarking Configurations for Reproducible Execution-Time Measurements of CI/CD-Style Workloads. 2026. (https://doi.org/10.1145/3838807).
4. Experimental Evaluation Methodology for the Era of No Steady Performance. 2026. (https://doi.org/10.1145/3839448).
5. Fray: An Efficient General-Purpose Concurrency Testing Platform for the JVM. 2025. (https://doi.org/10.1145/3764119).
6. Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads. 2024. (https://norma.ncirl.ie/8134/).
7. Virtual Threads after JDK 24: What Changed for Production Java. 2026. (https://www.infoq.com/articles/virtual-threads-after-jdk24/).
8. JEP 491: Synchronize Virtual Threads without Pinning. 2024. (https://openjdk.org/jeps/491).
9. Java Threading Options in 2026: Platform vs Virtual Threads. 2026. (https://www.jobrunr.io/en/blog/java-job-threading-options/).
10. Java 25 and the Evolution of Modern Java for Next-Generation Software Systems. 2026. (https://doi.org/10.55041/ijcope.v2i4.357).

11. eBPF jAgent: Transaction-level Resource Demand Tracing for Java Applications Using eBPF and OpenTelemetry. 2026. (`https://github.com/hm-green-it-lab/ssp2026`).

1.2.3 Виявлені проблеми

На основі аналізу наукової літератури, офіційних специфікацій OpenJDK та результатів інженерних досліджень виділено ключові проблеми диспетчеризації та надійності віртуальних потоків JVM:

1. **Залишкове блокування пулу потоків-носіїв на нативній межі (carrier-pool starvation).** Незважаючи на усунення моніторного пінінгу в JDK 24 (JEP 491), виконання нативного коду через C ABI (як у JNI, так і в Panama FFM API) та нативні зворотні виклики (callbacks) примусово утримує платформені потоки, спричиняючи приховане виснаження пулу ForkJoinPool при зростанні конкурентності.

2. **Відсутність замкнених аналітичних моделей масового обслуговування.** У науковій літературі бракує формалізованого математичного апарату теорії черг, який би дозволяв аналітично розраховувати критичний поріг насичення системи (λ_{crit}), довжину черг очікування та квантилі хвостових затримок для гібридних навантажень, що поєднують відмонтовувані операції вводу-виводу та невідмонтовувані нативні виклики.

3. **Ефект взаємного блокування задач та радіус ураження (blast radius).** Неізольоване закріплення потоків-носіїв під час тривалих нативних операцій блокує спільний пул ресурсів, що призводить до нелінійного падіння пропускної здатності та спалахів хвостових затримок (P_{99}) у сусідніх незалежних віртуальних потоках (зокрема неблокуючому мережевому I/O).

4. **Зміщення вузького місця та дефіцит архітектурних патернів ізоляції.** Неконтрольоване масштабування віртуальних потоків зміщує перевантаження на супутні підсистеми (пули з'єднань з базами даних, системні дескриптори), зумовлюючи каскадні відмови; водночас в інженерній практиці бракує верифікованих патернів захисту (семафорних бар'єрів-bulkheads, виділених пулів) для ізоляції нативного коду.

5. **Міжрівневий розрив спостережуваності рантайму та ядра ОС.** Планувальник JVM керує легковажними потоками у просторі користувача (M :

N), а операційна система – на рівні ядра (task_struct). Профайлери байткоду (JVMTI) не фіксують перемикання контексту ядра ОС (sched_switch, черги futex), тоді як системні трейсери (eBPF) прив'язані до потоків ядра та втрачають транзакційний контекст при виконанні віртуальних потоків.

6. **Ефект спостерігача та висока вартість багатофакторного тестування.** Глибоке системне трасування вносить значний оверхед у рантайм і спотворює вимірювання затримок, а повний емпіричний аналіз багатовимірного простору конфігурацій потребує надмірних ресурсів, що зумовлює потребу у легковажній телеметрії та сурогатних моделях машинного навчання.

1.3 Перевірені дослідницькі прогалини

1.4 Формування тем

1.4.1 Тема A (аналітична-теоритична)

1.4.2 Тема B (інженерна)

1.4.3 Тема C (міждисплінарна)

1.5 Порівняння тем

1.6 Peer review

1.6.1 Відгуки щодо теми A

1.6.2 Відгуки щодо теми B

1.6.3 Відгуки щодо теми C

1.7 Decision Matrix

Зведені показники порівняльного аналізу алгоритмів наведено в табл. 1.1. Зверніть увагу: табл. 1.1 займає більше однієї сторінки, демонструючи автоматичний механізм розриву.

Таблиця 1.1 – Результати комплексного порівняльного аналізу параметрів системи

№	Найменування тестового параметра	Метод А	Метод Б
1	Час первинної ініціалізації обчислювального середовища (мс)	14.2	9.8
2	Базовий обсяг оперативної пам'яті для кешування (МБ)	64.0	48.5

Продовження таблиці 1.1

№	Найменування тестового параметра	Метод А	Метод Б
3	Пропускна здатність вхідного потоку транзакцій (тис./с)	1250	1480
4	Затримка обробки критичних повідомлень під час піку (мс)	3.2	2.1
5	Рівень навантаження на обчислювальні ядра процесора (%)	78.4	62.1
6	Середнє питоме енергоспоживання вузла кластера (Вт)	46.2	38.0
7	Часова складність обробки у найкращому сценарії	$O(n)$	$O(1)$
8	Часова складність обробки в середньому сценарії	$O(n \log n)$	$O(n)$
9	Часова складність обробки у найгіршому сценарії	$O(n^2)$	$O(n \log n)$
10	Просторова складність алгоритмічної моделі	$O(n)$	$O(1)$
11	Стабільність збереження порядку пар еквівалентних ключів	Так	Ні
12	Рівень горизонтального масштабування обчислень	Високий	Помірний
13	Чутливість структури даних до фрагментації пам’яті	Низька	Середня
14	Частота влучань у дворівневий кеш процесора L1/L2 (%)	94.1	89.6
15	Швидкість деградації продуктивності при апаратних збоях	3.8%	6.5%
16	Кількість операцій порівняння на контрольний блок (тис.)	8.45	6.20
17	Кількість операцій перестановки на блок (тис.)	2.10	1.85
18	Час обробки масиву розмірністю $N = 10^4$ (мс)	4.2	3.1

Продовження таблиці 1.1

№	Найменування тестового параметра	Метод А	Метод Б
19	Час обробки масиву розмірністю $N = 10^5$ (мс)	48.6	35.8
20	Час обробки масиву розмірністю $N = 10^6$ (мс)	580.2	418.4
21	Час обробки попередньо впорядкованого масиву (мс)	12.1	8.2
22	Час обробки масиву, впорядкованого у зворотному напрямку (мс)	710.4	538.1
23	Вплив типу даних на загальну тривалість виконання	Низький	Помірний
24	Надійність збереження стану транзакцій при відмові (%)	99.99	99.95
25	Загальний приріст продуктивності після оптимізації	+34.5%	+41.8%

ВИСНОВОК